

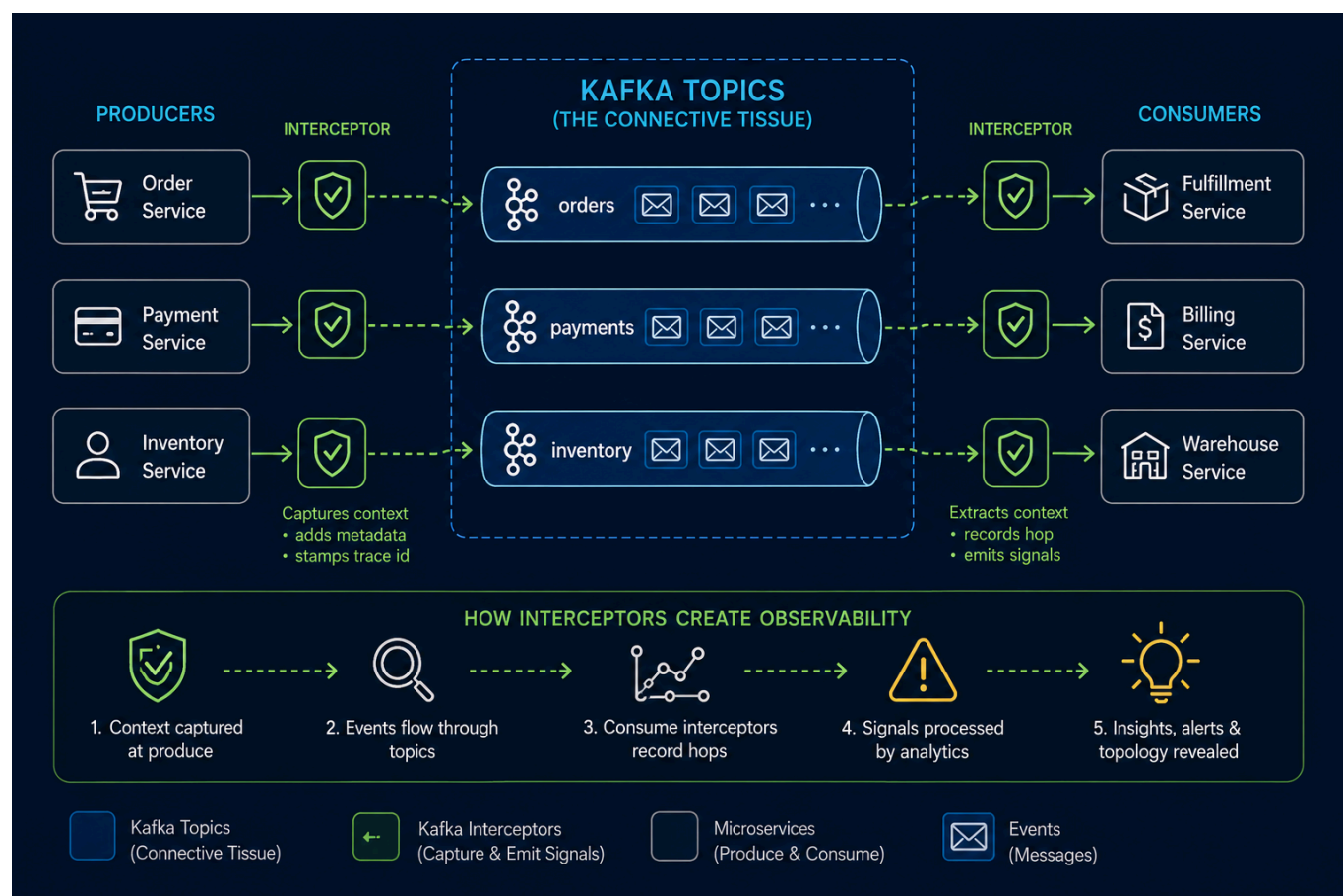
Confluent Kafka Isotope

Table of Contents

- [Purpose](#)
- [1.0 How an Isotope Traverses an Event Pipeline](#)
 - [1.1 Anatomy of the Isotope Tracing Artifact](#)
- [2.0 Architecture](#)
- [3.0 Getting Started](#)
 - [3.1 Integration Tests with Confluent Platform on Minikube](#)
 - [3.2 Seven Scalar Headers Flink SQL Reports with Apache Flink on Minikube](#)
 - [3.3 Seven Scalar Headers Flink SQL Reports with Confluent Cloud for Apache Flink](#)
 - [3.3.1 Why `latency\_percentiles` is a `ProcessTableFunction` \(PTF\)](#)
 - [3.3.2 \[OPTIONAL\] Eighth Report — AI Root-Cause Analysis \(RCA\)](#)
 - [3.4 \[OPTIONAL\] Prometheus Metrics Reporting with Grafana Visualization](#)
- [Resources](#)

Purpose

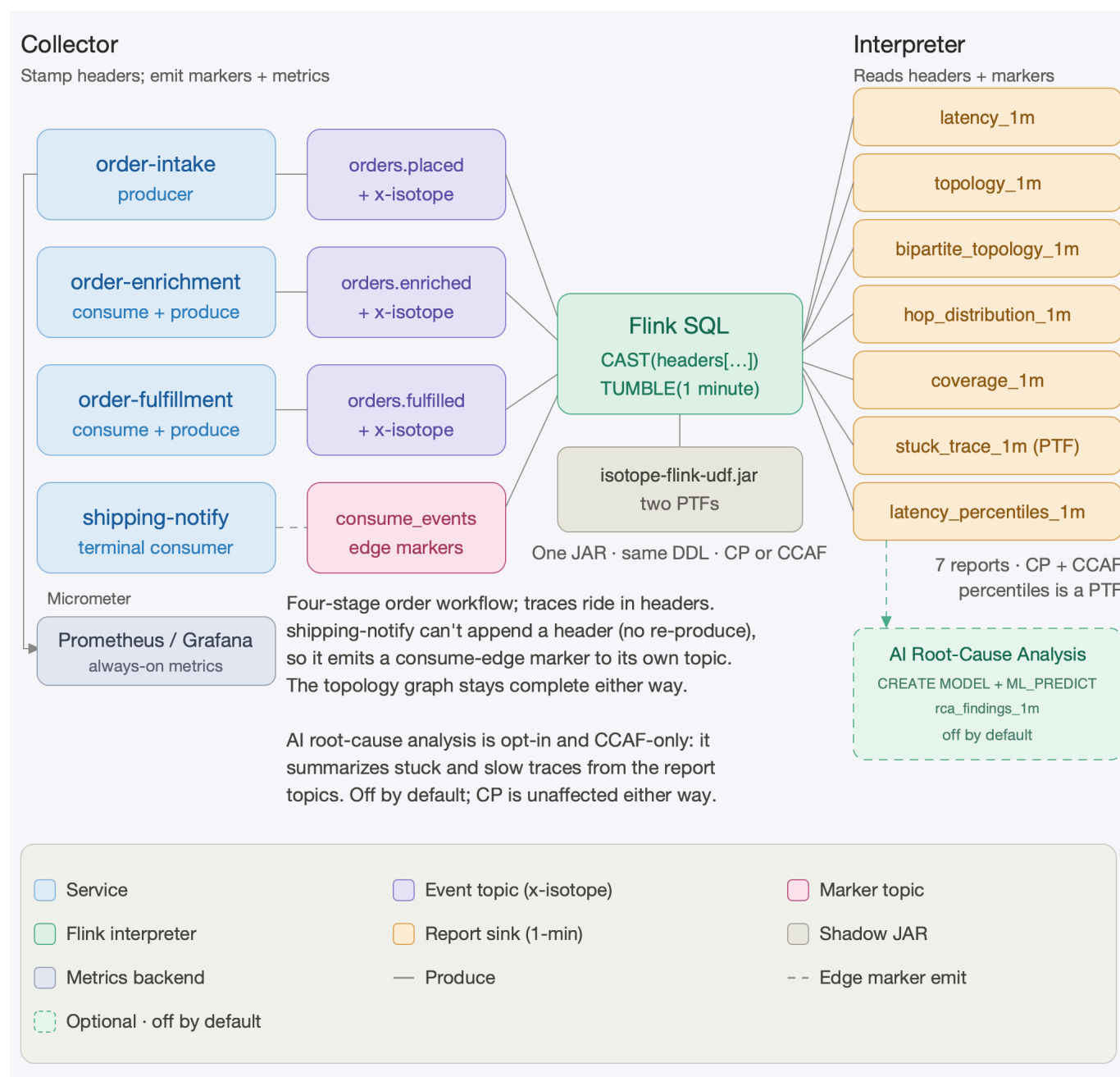
`confluent-kafka-isotope` is a reference implementation of an **e-commerce order pipeline that uses Kafka Interceptors to capture end-to-end event-tracing data, with Prometheus and Apache Flink analyzing and reporting that data in batch and near real time**, as visualized below.



Much like isotopes used to trace molecules through a biochemical pathway, each event carries lightweight metadata that allows it to be followed as it travels through Kafka topics and distributed microservices.

Kafka topics become the **connective tissue between services**, while **Kafka Interceptors** quietly **transform** the event pipeline itself into an **observable distributed system**.

This project demonstrates how **Kafka Interceptors become collectors** — inserting the isotope into record headers in place and, at terminal consumers, emitting consume-edge markers — while **Flink SQL serves as the interpreter**, reading those headers directly to produce seven 1-minute reports from a single JAR on both Confluent Platform and Confluent Cloud for Apache Flink (CCAF). Optionally, the producer interceptor can **emit Micrometer metrics to Prometheus as an always-on aggregate layer** alongside the per-trace Flink reports. (As depicted in the diagram below.)



With this approach, developers gain **end-to-end observability** into the flow of events through the Kafka-based microservices architecture, enabling both **real-time monitoring** and **post-hoc analysis** of event traces.

This end-to-end observability of the isotope tracing pipeline creates a **ladder of insights**, allowing developers to trace each event's journey, identify bottlenecks, and improve the performance and reliability of the entire system. The ladder is organized into **five tiers that answer 30 questions**, as visualized below.



- ▶ Easy — single-record, single-trace
- ▶ Easy to Medium — single per-minute aggregates
- ▶ Medium — cross-window deltas, anomalies, multi-report joins
- ▶ Hard — tail behavior, drift detection, correlated analysis
- ▶ Harder — forensic replay, compliance, cross-system joins

1.0 How an Isotope Traverses an Event Pipeline

An **Isotope** is a **lightweight tracing artifact attached to Kafka record headers**. Like a biochemical isotope used to trace molecules through a metabolic pathway, it allows the journey of a record through an event-driven architecture to be observed and analyzed.

This project models a Kafka pipeline as a **bipartite graph**: *services occupy one vertex set, topics the other, and every produce and consume operation forms an edge between them*. The resulting graph provides a unified view of the complete event topology, from producers to intermediate processing services to terminal consumers.

A **ProducerInterceptor** stamps each record with an isotope and appends one hop for every **send()** operation, capturing the produce edges. Consumers call **IsotopeContext.recordConsume()** to emit a lightweight marker representing the corresponding consume edges, while services that consume and then produce invoke **IsotopeContext.adoptFromRecord()** so the trace identity persists across every hop. Apache Flink reconstructs the complete **service → topic → service** graph from the isotope headers alone, producing seven reports: end-to-end latency, latency percentiles, produce-side topology, the full bipartite topology, hop distribution, per-topic coverage (a trace-loss funnel signal), and stuck-trace detection. The same implementation runs unchanged on both Confluent Platform (self-managed) with Confluent Manager for Apache Flink (CMF) and Confluent Cloud for Apache Flink (CCAF).

For more in-depth discussion of the **Isotope Tracing Design**, see [docs/design.md](https://docs.confluent.io/design/design.md).

1.1 Anatomy of the Isotope Tracing Artifact

The isotope is a **JSON object** that travels in the **x-isotope** Kafka record header, accompanied by **seven scalar headers** that Flink SQL reads directly. The JSON carries the trace identity, origin metadata, and full ordered hop history, while the scalar headers provide a flattened view of the most-recent-hop data for easier SQL access.

- **Header x-isotope** (JSON bytes) carries the full trace state and hop history, forwarded by every hop:
 - **t** — 16-byte **UUIDv7** trace ID ([RFC 9562](https://tools.ietf.org/html/rfc9562)): 48-bit millisecond timestamp in the high bits + 74 random bits. It remains stable for the life of the trace, and lexicographic byte order matches creation order — sorting trace IDs therefore produces chronological creation order. The random bits come from **ThreadLocalRandom**, not **SecureRandom**: a trace ID is a public observability identifier carried in Kafka headers, so the requirement is collision avoidance rather than unpredictability, without imposing **SecureRandom** overhead on every produced record.
 - **o** — origin timestamp (ms), identical to the timestamp embedded in the UUIDv7 trace ID; retained as a separate field for typed access from Flink SQL without decoding the UUID bytes.
 - **s** — origin service name, stamped once and forwarded unchanged for the life of the trace.
 - **p** — origin pipeline name (for example, **orders** vs. **location**); like **s**, stamped once at the origin and forwarded unchanged on every hop, allowing reports to slice traces by logical pipeline.
 - **h** — ordered list of hops, each represented as **{s: service, t: topic, m: tsMs}**.
 - **x** — **true** if the hop list exceeded **MAX_HOPS = 32** and the oldest hop was evicted.
- **Seven scalar headers** (UTF-8 strings) carry the most-recent-hop view, allowing Flink SQL to read them directly via **CAST(headers['x-isotope-...'] AS STRING)** without parsing the JSON hop array or requiring a UDF on either CCAF or CP Flink. See [scripts/flink/README.md](https://scripts.flink/README.md) for the complete scalar-header table.

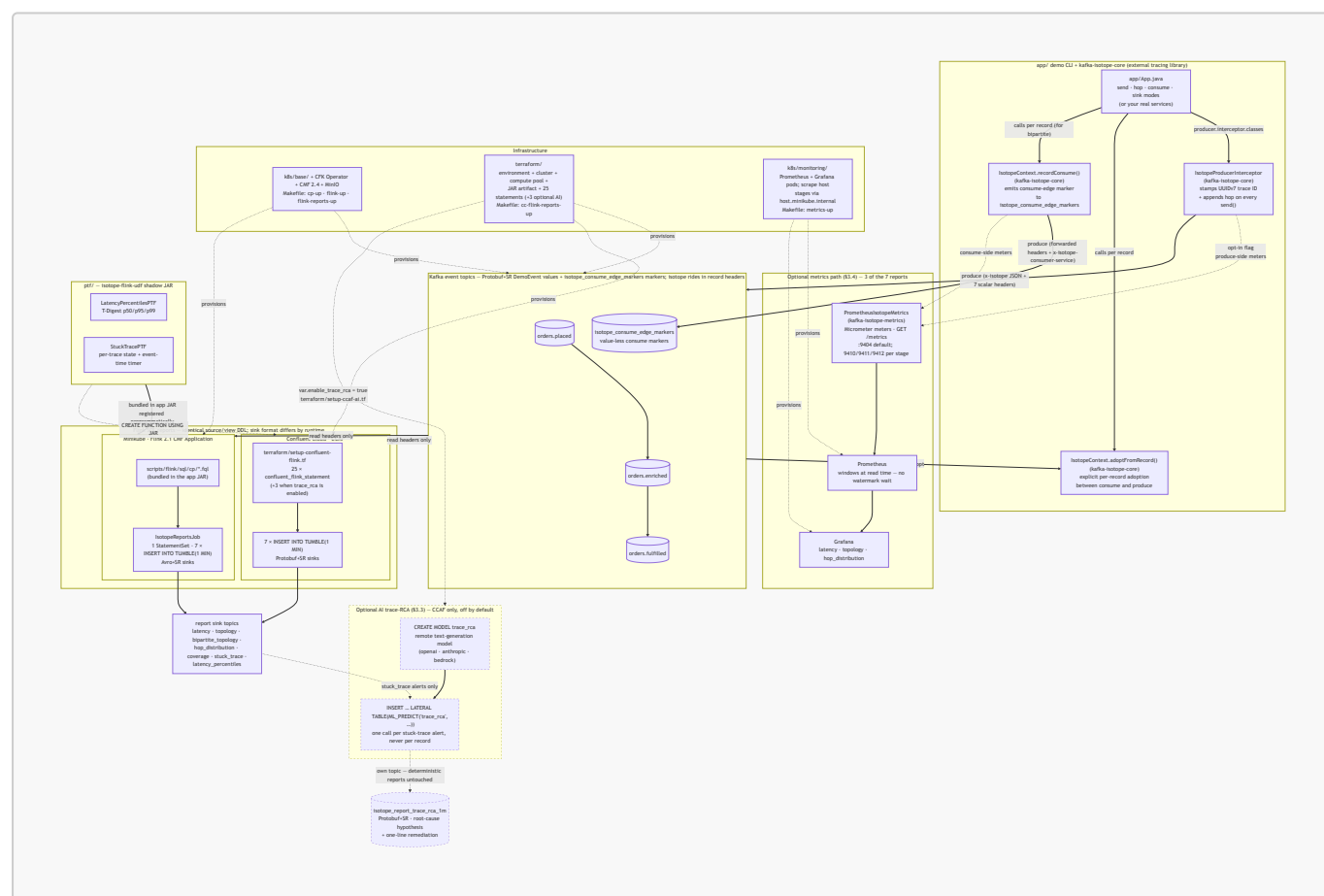
2.0 Architecture

A bird's-eye view of the moving parts. The demo CLI in `app/` consumes the external tracing library (`ai.signalroom:kafka-isotope-core`), which registers a Kafka `ProducerInterceptor` that stamps an isotope into record headers on every `send()`. Consume-then-produce services propagate the inbound trace by explicitly calling `IsotopeContext.adoptFromRecord(record)`. Business events then flow through a three-topic Kafka pipeline, where Flink SQL reads the isotope metadata and emits one-minute aggregate reports.

Both runtimes run the same seven logical reports off the same source and view definitions, though each runtime keeps its own copy of the SQL. **Confluent Platform (CP) + Flink** on Minikube executes the `.fql` files under `scripts/flink/sql/cp/` as a single Flink 2.1 Confluent Manager for Apache Flink (CMF) Application (`IsotopeReportsJob`), while **Confluent Cloud for Apache Flink (CCAF)** applies the same logical SQL as inline `confluent_flink_statement` Terraform resources under `terraform/`. The shadow JAR from `ptf/`—which powers two of the seven reports—runs unchanged on both runtimes: bundled into the CP application JAR and uploaded as a Flink artifact on CCAF.

Alongside that—**additive, opt-in, and disabled by default**—the interceptor can also emit **Micrometer** metrics for **Prometheus**, with **Grafana** providing visualization ([§3.4 Prometheus Metrics Reporting with Grafana Visualization](#)). This path produces the three **stateless** reports (`latency_1m`, `topology_1m`, and `hop_distribution_1m`) without requiring a stream processor. The remaining four reports continue to run in Flink because they depend on per-`trace_id` state or absence-of-event analysis, which falls outside Prometheus's query model.

(Kafka is drawn once below for brevity; each runtime provisions its own Kafka cluster.)



► See Repo Layout

3.0 Getting Started

First, clone the repo to your local machine using the [GitHub CLI](#):

```
gh repo clone j3-signalroom/confluent-kafka-isotope
```

Change to the repo directory:

```
cd /path/to/confluent-kafka-isotope
```

Then decide how you want to run the repo:

3.1 Integration Tests with Confluent Platform on Minikube

```
make install-prereqs      # docker, kubectl, minikube, helm, gettext,
gradle, openjdk17
make check-prereqs       # verify they're on PATH
```

Default Minikube sizing (override via env): `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480`, `MINIKUBE_DISK=50g`.

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start      # one-time
make cp-up               # CFK Operator +
Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up         # localhost:30092 → Kafka,
localhost:8081 → Schema Registry
```

Then run the suite:

```
./gradlew :app:integrationTest      #
every IT below
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT'  #
just one
```

Override the endpoints if needed:


```
./gradlew :app:integrationTest \
-PkafkaBootstrap=localhost:30092 \
-PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
BrokerSmokeIT	AdminClient can create/list/delete a topic via the NodePort port-forward
ProducerInterceptorIT	A consumer sees the <code>x-isotope</code> JSON header + all 7 scalar reporting headers with the expected origin/hop values, and the Protobuf round-trip preserves <code>DemoEvent.source / payload</code>
ThreeStageHopPropagationIT	<code>order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service</code> produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = <code>order-intake-service</code> , this = <code>order-enrichment-service</code> , hop count = 2) at the terminal; consume-then-produce hops use <code>IsotopeContext.adoptFromRecord</code> to carry the trace forward
BipartiteTopologyIT	The 4-stage <code>order-intake-service → topic-AB → order-enrichment-service → topic-BC → order-fulfillment-service → topic-CD → shipping-notification-service</code> chain emits exactly three consume-edge markers to a per-test markers topic — one per consume edge. Every marker carries the trace ID, forwarded <code>x-isotope-*</code> scalars describing the upstream producer, and the new <code>x-isotope-consumer-service</code> naming the downstream consumer. Asserts the <code>(consumer_service, consumed_topic)</code> set is exactly the three pairs of stages 2-4

3.2 Seven Scalar Headers Flink SQL Reports with Apache Flink on Minikube

Caveat: Minikube is a single-node cluster. It is not a production-like environment, but it is sufficient for local development and testing. The Confluent Platform (CP) + Flink stack runs in Minikube, and you can port-forward Kafka and Schema Registry to your local machine.

To **run**, **test**, and **debug** Apache Flink like a production engineer, this project provides a full Confluent Platform + Flink stack running locally on [Minikube](#) — no cloud required.

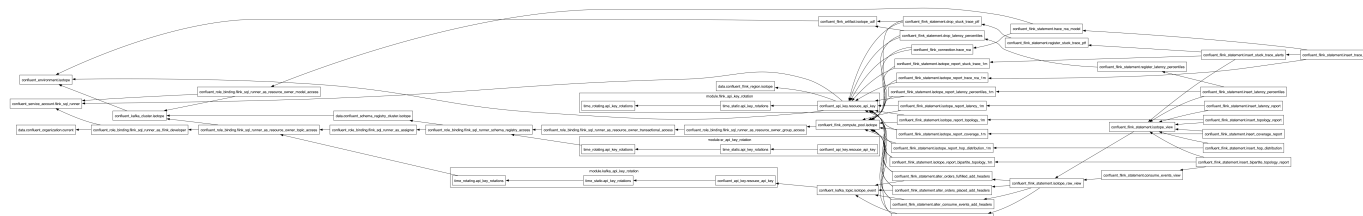
- **Confluent Platform** (KRaft mode) via Confluent for Kubernetes (CFK)
- **Apache Flink 2.1.2** via the Confluent Flink Kubernetes Operator 1.140.1
- **Confluent Manager for Apache Flink (CMF) 2.4.0** for Flink environment management

Resource	Default
CPUs	6
Memory	20 GB
Disk	50 GB

Seven reports — five pure Flink SQL plus two JAR-backed PTFs — run as a **single Flink 2.1 CMF Application** (`IsotopeReportsJob`, one `StatementSet`, seven sinks) submitted through CMF and executed by the Confluent Flink Kubernetes Operator. No raw session cluster is involved; `k8s/base/flink-cluster-deployment.yaml` (`make flink-deploy` / `make flink-sql`) is a separate, optional path for ad-hoc SQL. Confluent Cloud runs the same seven reports from its own copy of the SQL, inlined in Terraform — see [§3.3 Confluent Cloud for Apache Flink](#) for that path; this section is the local-Minikube one.

Report sink topics ride **Avro+SR** (`avro-confluent`, auto-registered on first write) so Control Center renders them natively — a deliberate *format-by-domain* split: app events are **Protobuf+SR** (`DemoEvent`), Flink aggregates are **Avro+SR** (cp-flink ships no SR-integrated Protobuf format), and the consume-edge marker topic `isotope_consume_edge_markers` is **null-value / headers-only**. Not a defect — a clean split by domain.

The Confluent Cloud for Apache Flink (CCAF) parallel of [§3.2 CP + Apache Flink on MiniKube](#), driven by Terraform under [terraform/](#). The Terraform graph below depicts the resources deployed in Confluent Cloud.



`make cc-flink-reports-up` provisions a fresh Confluent Cloud environment containing:

- Kafka cluster
- Compute pool
- Two rotating service-account API key pairs (Kafka + Schema Registry)
- The uploaded PTF JAR
- 25 `confluent_flink_statement` resources (4 `ALTER TABLE` + 3 `VIEW` + 7 sink `CREATE TABLE` + 2 `CREATE FUNCTION` + 7 `INSERT INTO`, plus 2 transient `DROP FUNCTION`)

The full provision → deploy → traffic → teardown sequence is consolidated in [docs/runbook-ccaf.md](#). `make cc-flink-reports-up` (~6–8 min first run; idempotent re-applies), drive traffic with `scripts/cc-app-run.sh place|enrich|fulfill|ship` across **multiple** 1-minute windows (a single burst sits in one open window forever, and you wait ~90s after the last record), then `make cc-flink-reports-down` to `terraform destroy` the whole environment.

Format-by-runtime (not-by-domain). CP's reports land on **Avro+SR** (`'value.format' = 'avro-confluent'` in `scripts/flink/sql/cp/05_report_sinks.fql`). CCAF's reports land on **Protobuf+SR** (`'value.format' = 'proto-registry'` in each sink's `WITH` clause in `terraform/setup-confluent-flink.tf`). The two runtimes' SQL is otherwise unshared: CP's lives hardcoded in `scripts/flink/sql/cp/`, CCAF's lives inline as `confluent_flink_statement` resources in `terraform/setup-confluent-flink.tf`.

3.3.1 Why `latency_percentiles` is a `ProcessTableFunction` (PTF)

Since CCAF does not support [User-Defined AGGregate functions \(UDAGG\)](#), I implemented the percentiles report as a `ProcessTableFunction` to keep it portable across runtimes.

`LATENCY_PERCENTILES` (class `LatencyPercentilesPTF`) performs its own 1-minute tumbling-window aggregation over a T-Digest sketch using per-window state and event-time timers. **A PTF avoids that UDAGG restriction, so it registers and runs on both runtimes, just like `STUCK_TRACE_PTF`. Both runtimes therefore run the same seven reports: `latency` (avg/min/max), `topology` (produce-side), `bipartite_topology` (full service↔topic↔service graph), `hop_distribution`, `coverage`, `stuck_trace`, and `latency_percentiles` (p50/p95/p99).**

3.3.2 [OPTIONAL] Eighth Report — AI Root-Cause Analysis (RCA)

Beyond the seven deterministic reports, an **eighth, AI-generated report** turns each stuck-trace *alert* into a natural-language root-cause hypothesis plus a one-line remediation. This **CCAF-only** feature is wired by `terraform/setup-ccaf-ai.tf` and can be enabled **at deploy time** by setting `ENABLE_TRACE_RCA=true` and supplying the other required **RCA** arguments to the `make cc-flink-reports-up` target.

When enabled, it adds three `confluent_flink_statement` resources:

- `CREATE MODEL trace_rca` — registers a remote text-generation model.
- A `proto-registry` sink table `isotope_report_trace_rca_1m`.
- An `INSERT ... SELECT ... LATERAL TABLE(ML_PREDICT('trace_rca', ...))` that calls the model **once per alert** (reading the low-volume 1-minute stuck-trace report topic, never the source stream) and writes the hypothesis to its own topic — it never overwrites the deterministic reports.

The default provider is OpenAI (`gpt-4o`). Claude is supported via **AWS Bedrock** (`rca_model_provider = "bedrock"` with AWS credentials). Other supported providers: `vertexai`, `azureopenai`, `googleai`, `sagemaker`, `azureml`.

3.4 [OPTIONAL] Prometheus Metrics Reporting with Grafana Visualization

Three of the seven reports — `latency_1m`, `topology_1m`, `hop_distribution_1m` — are pure stateless scalar aggregation over bounded-cardinality dimensions (service / topic / hop_count, never `trace_id`). Those don't need a stream processor: the `producer interceptor` already has every value in scope on each `send()`, so it emits them as **Micrometer** meters (`PrometheusIsotopeMetrics.java`) and lets **Prometheus** window them at read time, with **Grafana** on top. The other four (`latency_percentiles`, `coverage`, `bipartite_topology`, `stuck_trace`) are per-`trace_id` stateful or absence-of-event problems Prometheus can't express, so they **stay in Flink**. This path is **additive and opt-in** — a 3-Micrometer / 4-Flink split.

Full details — the meter/PromQL reference, the produce- and consume-side signals, the two deliberate gaps (`distinct_traces`, windowed `min`), and why percentiles stays a PTF — are in [docs/metrics.md](#). The one-command Prometheus + Grafana showcase has its own runbook: [k8s/monitoring/README.md](#) (`make metrics-up`).

Resources

- [Medium Article: Kafka's quiet observability superpower — Kafka Interceptors](#)